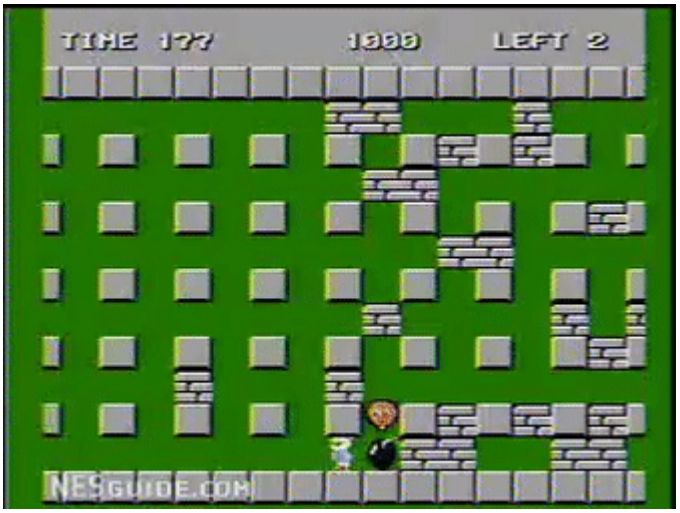
LSP kursa projekts 2026

Spēles izvēle

Pēc diskusijām un balsojuma tika izvēlēta spēle Bomberman.

Spēles specifikācija

- Wikipedia: Bomberman (1983 video game) [kā reference, nevis galējais variants]
- ttps://www.youtube.com/watch?v=knNr-7fExCE [*gameplay footage* iedvesmai: Playforia Bombermania]
- https://www.retrogames.cz/play_085-NES.php [tiešsaistes versija iedvesmai]



Spēles prasības4

- spēlētāja pārvietošanās pa režģveida karti,
- bumbu novietošana un sprādzieni pēc laika ar noteiktu darbības rādiusu,
- bonusu laukumi, kurus atklājot, tie modificē spēlētāja un sprādzienu īpašības,
- iznīcināmi un neiznīcināmi bloki,
- spēlētāja bojāeja, nonākot sprādzienā,
- uzvaras un spēles beigu nosacījumi,
- vairāku spēlētāju atbalsts tīklā,
- spēles stāvoklis tiek atjaunots 20 reizes sekundē (ticks).

Kā uzsākt spēli?

Katrs serveris atbalsta vienu “spēles telpu”, t.i., uz viena servera nevar vienlaicīgi noritēt vairākas spēles. Istabā var būt līdz 8 lietotājiem, no kuriem visi piedalās spēlē. Serveris sākas priekšnama režīmā, kurā var sagaidīt spēlētāju pievienošanos. Lai spēle sāktos, istabas izveidotājam (pirmajam spēlētājam) jānorāda laukuma konfigurācija no servera dotajām izvēlēm.

Spēle sākas, kad pirmais spēlētājs izvēlas “Sākt”.

Pēc spēles beigām serveris atkal ieiet priekšnamā.

Serveris nodod klientam sekojošu informāciju par katru laukuma konfigurācijas izvēli (karti un spēlētāju skaitu):

- laukuma izmērs,
- spēlētāju skaits (vai skaita robežas: min un max),
- laukuma izkārtojums (kur un kādas šūnas pastāv uzsākot spēli).

Katrs spēlētājs sāk ar šādām noklusējuma īpašībām:

- pieejamo bumbu skaits ir 1,
- bumbu sprādziena rādiuss 1,
- bumbu atskaites laiks ir 3 sekundes,
- spēlētāja kustības ātrums ir 3 šūnas sekundē. Kustība ir monotona.

Spēles darbības (gājieni un vai aktivitātes)

Spēle notiek taisnstūrveida spēles laukumā – divdimensionālā šūnu matricā. Katra šūna var būt vienā no šādiem stāvokļiem:

- tukšums,
- neiznīcināma siena,
- iznīcināms bloks,
- bumba,
- sprādziena zona,
- bonusu saturoša šūna.

Spēlē piedalās 2 līdz 8 spēlētāji. Katram spēlētājam ir:

- unikāls lietotājvārds,
- pozīcija laukumā,
- dzīvības statuss,
- bumbu sprādziena rādiuss (mantots no spēlētāja),
- bumbu atskaites laiks (mantots no spēlētāja),
- pieejamo bumbu skaits,

- spēlētāja kustības ātrums.

Spēles laukums tiek ielasīts no teksta faila. Tajā sākotnēji tiek izvietoti:

- neiznīcināmās sienas noteiktā regulārā rakstā,
- iznīcināmie bloki nejauši vai pēc iepriekš definētas kartes,
- brīvas sākuma zonas pie katra spēlētāja sākuma pozīcijas, lai spēlētājs nevarētu uzreiz iesprūst.

Bumbu drīkst novietot tikai uz tās šūnas, **kurā pašlaik atrodas spēlētājs**. Vienā šūnā vienlaikus var atrasties ne vairāk kā viena bumba. Bumbai ir taimeris līdz sprādzienam.

Sprādziens notiek krusta veidā līdz attālumam ***t***, ja vien to neaptur šķērslis: neiznīcināma siena pilnībā aptur sprādzienu, iznīcināms bloks tiek iznīcināts, bet sprādziens tālāk neiet cauri šai šūnai, cita bumba, ko sasniedz sprādziens, detonē nekavējoties.

Spēlētājs, kas atrodas sprādziena zonā sprādziena brīdī, zaudē dzīvību. Spēlei jāatbalsta variants, kad ir vismaz viena dzīvība. Var realizēt arī variantu ar vairākām dzīvībām. Kad ir visas dzīvības ir zaudētas, spēlētājs var novērot spēli, bet nevar vairs piedalīties spēlē un spēlētāja tēls tiek izņemts no laukuma.

Spēlētājs kontrolē tēlu reāllaikā. Spēlētāja darbības spēles laikā var būt šādas:

- pārvietoties vienu šūnu uz augšu,
- pārvietoties vienu šūnu uz leju,
- pārvietoties vienu šūnu pa kreisi,
- pārvietoties vienu šūnu pa labi,
- novietot bumbu.

Spēlētājs nedrīkst ieiet šūnā, kurā atrodas:

- neiznīcināma siena,
- iznīcināms bloks,
- bumba (drīkst atrasties tajā pašā šūnā, kad to novieto, bet ***atpakaļ*** ieiet nevar)
- cits spēlētājs

Bonusu saturošās šūnas iespējamie efekti:

- spēlētāja pārvietošanās ātruma palielināšana +1,
- bumbas sprādziena rādiusa palielināšana +1,
- bumbas atskaites laika palielināšana +1.

Spēles beigu un uzvaras nosacījumi

Spēle beidzas, kad dzīvs paliek tikai viens spēlētājs. Šis spēlētājs tiek pasludināts par uzvarētāju. Ja raundam ir noteikts maksimālais ilgums, tad pēc laika atskaites beigām tiek pasludināts neizšķirts spēles iznākums un parādīta statistika, kas ļautu kaut kādā mērā salīdzināt spēlētāju veikumu:

- izsisto pretinieku skaits,
- iznīcināto bloku skaits,
- savākto bonusu skaits.

Spēles laukuma konfigurācijas faila sintakse. Pirmajā rindā:

- Laukuma izmērs (rindas, kolonas), piemēram: 11 13
- Splētāja ātrums (bloki/ticks),
- Sprādziena efekta bīstamības laiks (ticks).
- Sprādziena attālums (bloki)
- Sprādziena atskaites laiks (ticks). Lai autors var aizmukt.

Seko laukuma karte. Elementi katrā laukuma šūnā:

- H cietais bloks,
- S mīkstais bloks,
- 1..8 spēlētājs ar attiecīgu numuru,
- B bumba,
- A spēlētāja pārvietošanās ātruma palielināšana +1,
- R bumbas sprādziena rādiusa palielināšana +1,
- T bumbas atskaites laika palielināšana +1.

Piemērs konfigurācijai failā:

```
6 8 3 5 3 4
H 1 . . . . A . .
. . S S . R . . .
T . . 2 . . . . .
. . . . . . . . .
. . . . . . . . .
. . . . . . . . .
```

Komunikāciju protokols

Zema līmeņa nosacījumi

Komunikācija notiek ar TCP binārā formātā, skaitļi tiek sūtīti kā big-endian (lai tos apstrādātu, var izmantot `byteorder(3)` vai `endian(3)` funkcijas). Katra ziņa sastāv no:

- `uint8_t`, kas norāda ziņas tipu
- `uint8_t`, kas norāda:
    - spēlētāja ID, uz kuru ziņa attiecas. (piemēram, ja serveris nosūta klientam `WELCOME` ziņu, tas ir jaunā spēlētāja ID). Šis skaitlis tiks turpmāk saukts par ziņas mērķi.
    - Ja tas nav attiecināms uz spēlētāju (piemēram, klientam kaut ko sūtot serverim), tad tā vērtība tiek iestatīta kā 255. Pārsūtot ziņas, serveris vienmēr iestata šo skaitli uz klienta, kurš ziņu sākotnēji nosūtījis, ID.
    - Alternatīva ir vērtība 254, kas norāda ka ziņa ir visiem klientiem (broadcast).
- ziņas satura jeb *payload*.

Saturs sastāv no sekojošiem datu tipiem, kas ziņu aprakstā tiks apzīmēti šādi:

- `uint8_t, uint16_t, uint32_t, uint64_t` – tiek sūtīti kā atbilstoša lieluma (8, 16, 32, 64) bezzīmes skaitļi
- `int8_t, int16_t, int32_t, int64_t` – tiek sūtīti kā atbilstoša lieluma *two's complement* skaitļi
- `bool` – tiek sūtīts tāpat kā `uint8_t`, bet 0 tiek uztverta kā aplama būla vērtība un 1 tiek uztverts kā patiesa būla vērtība

Protokola struktūra, ziņu vispārējais formāts

Datu struktūra, kas kopīga visām ziņām.

```
typedef struct {
      uint8_t msg_type;  // ziņas tips, kas nosaka datu struktūru
      uint8_t sender_id;
      uint8_t target_id; // adresāta ID. 255=server. 254=broadcast.
      // payload
} msg_generic_t;
```

"*`Payload`*" var saturēt citas datu struktūras, vai arī tikt izlaists īsām ziņām. Var arī saturēt mainīga garuma simbolu virkni, kurai priekšā ir `uint16_t len` - virknes garums, neieskaitot len.

Turpmāk seko datu struktūras katram ziņu tipam. Zemāk pieejama arī galvenes (*header*) datne.

Ziņu tipi un datu lauki

Spēles statuss	
0	Priekšnams

1	Spēle notiek
2	Beigu ekrāns

00 tips	HELLO pārsūtāma	
Klienta identifikators char[20]		Spēlētāja vārds char[30]
Šo ziņu klients nosūta serverim tūlīt pēc pieslēgšanās. Klienta identifikators ir simbolu virkne, kas identificē klienta programmas nosaukumu un versiju, savukārt spēlētāja vārds unikāli identificē pašu spēlētāju. Kad serveris šo ziņu saņem, tas izvērtē, vai ir gatavs pieņemt klientu. Ja nav, tad sūta DISCONNECT ziņu, ja ir, tad šo ziņu pārsūta pārējiem klientiem un attiecīgajam klientam nosūta WELCOME ziņu.		

01 tips	WELCOME		
Servera identifikators `char[20]`	Spēles statuss `uint8_t`	Garums `uint8_t`	Pārējie klienti `(uint8_t, bool, char[30])[]`
Šī ziņa vienmēr tiek sūtīta kā atbilde uz `HELLO` ziņu. Ja klients, kas mēģina pieslēgties, 30 sekunžu laikā nesaņem ne šo ziņu, ne `DISCONNECT` ziņu, tam vajadzētu aizvērt TCP savienojumu. Pirms šī ziņa ir saņemta, citas ziņas klients sūtīt nedrīkst. ● Servera identifikators, tāpat kā klienta identifikators, identificē servera programmas nosaukumu un versiju. ● Spēles statuss ir viens no skaitļiem augstāk redzamajā statusu tabulā. Tas norāda to, vai spēle ir lobijā vai arī ir sākusies. ● Visbeidzot, tiek nosūtīts masīvs ar pārējo klientu ID, spēlētāju gatavību un spēlētāju vārdiem. Ja spēle nav lobija stāvoklī, tad klients ir gatavs tad un tikai tad, ja tas piedalās spēlē. Ja statuss ir 1, tad serverim vajadzētu arī nosūtīt `SYNC_BOARD` ziņu par katru spēlētāju, savukārt, ja statuss ir 3, tad serverim vajadzētu nosūtīt `WINNER` ziņu. Kā ziņas avots tiek norādīts piešķirtais spēlētāja ID.			

05 tips	LEAVE pārsūtāma
Ja klients šo ziņu nosūta serverim, tas nozīmē, ka tas grasās atvienoties. Pēc šīs ziņas nosūtīšanas klientam jāaizver savienojums. Ja serveris šo ziņu nosūta klientam, tad tas nozīmē, ka klients, kura ID ir norādīts kā ziņas avots, ir atvienojies vai ticis atvienots.	

02 tips	DISCONNECT
Ziņu nosūta serveris klientam, pirms close(socket). Pēc šīs ziņas saņemšanas klientam vajadzētu aizvērt TCP savienojumu.	

03 tips	PING

Ziņu var nosūtīt gan klients, gan serveris otrai komunikācijas pusei. Ja otra puse 30 sekunžu laikā neatbild ar PONG ziņu, var pieņemt, ka tai ir iestājies timeouts.

04 tips	PONG
Atbilde uz PING ziņu.	

06 tips	ERROR
Kļūdas paziņojums char[]	
Ja kādu klienta nosūtīto ziņu neizdevās īstenot un ziņa ir pārsūtāma (izņemot HELLO ziņu) vai uz to ir paredzēta cita veida atbilde, tiek nosūtīta šī ziņa. Iespējama klienta atbilde uz šo ziņu varētu būt spēles atjaunošana pareizā stāvoklī ar REQ_SYNC ziņu.	

10 tips	SET_READY pārsūtāma
Klients šo ziņu serverim nosūta, lai norādītu savu gatavību sākt spēli. Kad visi spēlētāji ir iestatījuši sevi kā gatavus, tad tiek sākta spēle. Serveris ir tiesīgs nosūtīt šo ziņu arī tad, ja attiecīgais klients nav tādu nosūtījis, tādējādi “piespiedu kārtā” padarot to par spēlētāju. Tā var īstenot, piemēram, iespēju pieslēgties pēc savienojuma pazušanas spēles laikā.	

20 tips	SET_STATUS pārsūtāma
Spēles statuss uint8_t	
Serveris šo ziņu nosūta klientiem, lai paziņotu par spēles stāvokli, kā norādīts augstāk esošajā tabulā. Ja šis statuss ir 1 (spēle notiek), tad serverim jānosūta arī SYNC_BOARD ziņa par katru spēlētāju, savukārt ja tas ir 3 (beigu ekrāns), tad serverim jānosūta arī WINNER ziņa. Šī ziņa vienmēr tiek nosūtīta, kad statuss mainās, vai arī kā atbilde uz REQ_SYNC ziņu. Jebkurš klients var šo ziņu nosūtīt tad un tikai tad, ja pašreizējais spēles statuss ir 3 (uzvara) un iestatāmais spēles statuss ir 0 (lobijs), lai mainītu spēles statusu.	

23 tips	WINNER

Uzvarētāja ID `uint8_t`
Šo ziņu serveris nosūta ikvienam spēlē iesaistītam klientam, lai paziņotu par to, kurš spēlētājs uzvarēja spēlē.

07 tips	MAP
Kartes izmērs `uint8_t, uint8_t (H,W)`	Kartes aizpildījums `uint8_t * H * W`
Šo ziņu serveris sūta klientiem ar tekošo kartes informāciju. W=width, H=height. Katrai šūnai viens baits, līdzīgi kā kartes konfigurācijas failā.	

30 tips	MOVE_ATTEMPT
	Kustības virziens `uint8_t`
Šo ziņu klients sūta serverim, ja viņš vēlas paiet kaut kādā virzienā. Viņš nosūta savu spēlētāja ID un kustības virzienu, kurā viņš vēlas iet. Ja tur var iet, tad serveris nosūta tālāk visiem klientiem “MOVE”. Kustības virziens ir kodēts kā ASCII simbols: U-up, D-dowm, L-left, R-right.	

40 tips	MOVED pārsūtāma
Spēlētāja ID `uint8_t`	Jaunā atrašanās koordināte `Uint16_t`
Šo ziņu serveris sūta visiem klientiem, lai informētu, ka kāds spēlētājs ir pakustējies uz jaunu šūnu.	

31 tips	BOMB_ATTEMPT
Spridzekļa šūna `uint16_t`	
Šo ziņu klients sūta serverim, ja viņš vēlas nolikt spridzekli. Serveris pārbauda, un, ja var, nosūta visiem klientiem “BOMB” paketi.	

41 tips	BOMB pārsūtāma			
BumSpēlētāja ID uint8_t	Spridzekļa šūna uint16_t			
Šo ziņu serveris sūta klientiem, lai parādītu, ka ir novietots spridzeklis.				

42 tips	EXPLOSION_START pārsūtāma			
Rādiuss uint8_t	Spridzekļa šūna uint16_t			
Šo ziņu serveris sūta klientiem, lai parādītu, ka notiek sprādziena sākums.				

43 tips	EXPLOSION_END pārsūtāma			
	Spridzekļa šūna uint16_t			
Šo ziņu serveris sūta klientiem, lai parādītu, ka notiek sprādziena beigas.				

44 tips	DEATH pārsūtāma			
Spēlētāja ID uint8_t				
Šo ziņu serveris sūta klientiem, lai parādītu, ka ir spēlētājs ir miris (atradies sprādziena rādiusā).				

45 tips	BONUS_AVAILABLE pārsūtāma			
Bonusa veids uint8_t	Bonusa šūna uint16_t			

Šo ziņu serveris sūta klientiem, lai parādītu, ka pieejams bonusa laukums.

46 tips	BONUS_RETRIEVED pārsūtāma			
Spēlētāja ID uint8_t	Bonusa šūna uint16_t			
Šo ziņu serveris sūta klientiem, lai parādītu, ka spēlētājs uzkāpis uz bonusa (ieguvis bonusu). Bonusa veids klientiem jau zināms kopš BONUS_AVAILABLE. Sūta visiem, lai zinātu, ka tas vairs nav pieejams (un spētu to atbilstoši (ne)attēlot).				

47 tips	BLOCK_DESTROYED pārsūtāma			
Bloka šūna uint16_t				
Šo ziņu serveris sūta klientiem, lai parādītu, ka mīkstais bloks ir iznīcināts.				

Kopīgā galvenes (*header*) datne:

```
config.h

#define MAX_PLAYERS 8
#define TICKS_PER_SECOND 20

typedef enum {
      GAME_LOBBY = 0,
      GAME_RUNNING = 1,
      GAME_END = 2
} game_status_t;

typedef enum {
      DIR_UP = 0,
      DIR_DOWN = 1,
      DIR_LEFT = 2,
      DIR_RIGHT = 3
} direction_t;

typedef enum {
      BONUS_NONE = 0,
```

```
    BONUS_SPEED = 1,
    BONUS_RADIUS = 2,
    BONUS_TIMER = 3
} bonus_type_t;

typedef enum {
    MSG_HELLO = 0,
    MSG_WELCOME = 1,
    MSG_DISCONNECT = 2,
    MSG_PING = 3,
    MSG_PONG = 4,
    MSG_LEAVE = 5,
    MSG_ERROR = 6,
    MSG_SET_READY = 10,
    MSG_SET_STATUS = 20,
    MSG_WINNER = 23,
    MSG_MOVE_ATTEMPT = 30,
    MSG_BOMB_ATTEMPT = 31,
    MSG_MOVED = 40,
    MSG_BOMB = 41,
    MSG_EXPLOSION_START = 42,
    MSG_EXPLOSION_END = 43,
    MSG_DEATH = 44,
    MSG_BONUS_AVAILABLE = 45,
    MSG_BONUS_RETRIEVED = 46,
    MSG_BLOCK_DESTROYED = 47
} msg_type_t;


typedef struct {
    uint8_t id;
    char name[MAX_NAME_LEN + 1];
    uint16_t row;
    uint16_t col;
    bool alive;
    bool ready;

    uint8_t bomb_count;
    uint8_t bomb_radius;
    uint16_t bomb_timer_ticks;
    uint16_t speed;
```

```
} player_t;

typedef struct {
	bool active;
	uint8_t owner_id;
	uint16_t row;
	uint16_t col;
	uint8_t radius;
	uint16_t timer_ticks;
} bomb_t;

static inline uint16_t make_cell_index(uint16_t row, uint16_t col,
uint16_t cols) {
	return row * cols + col;
}
```